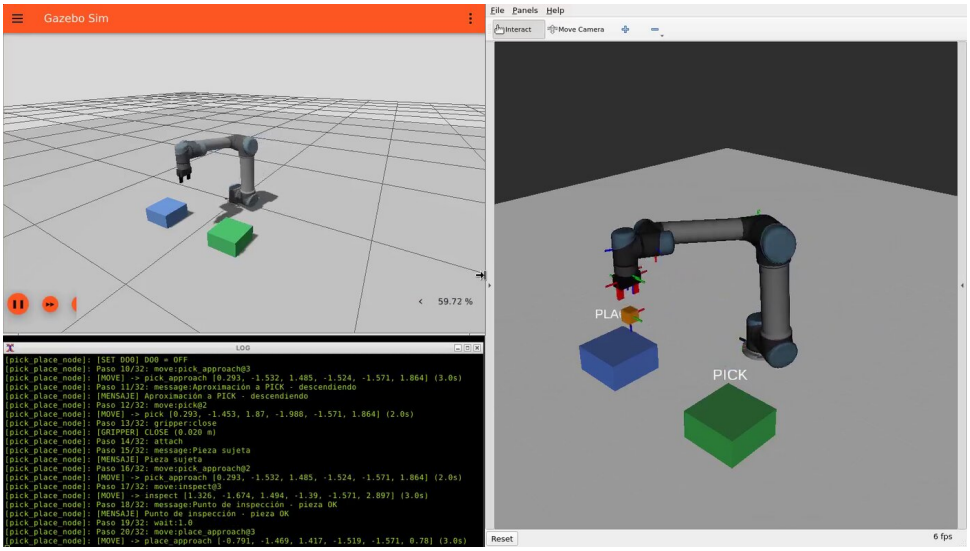
Laboratorio de Robótica

Simulación de un UR5 con gripper en ROS 2: control por tópicos, rutina Pick & Place y planificación con MoveIt 2

Integrantes

Tomás
Daniela Molina
Santiago Chacón



25 de octubre de 2026

Índice

1. Enlaces de entrega **2**

2. Objetivos **2**

3. Entorno y metodología **3**

4. Actividad 1: simulación y control por tópicos **3**

5. Actividad 2: rutina Pick & Place **5**

5.1. Diseño 5

5.2. Puntos de interés y operaciones 6

5.3. Resultados 6

6. Actividad 3: paquete MoveIt con el Setup Assistant **7**

6.1. Configuración 7

6.2. Paquete obtenido 10

6.3. Pruebas de planificación en RViz 10

7. Actividad 4: ejecución de trayectorias con MoveIt **11**

8. Hallazgos, dificultades y alternativas **12**

9. Conclusiones **13**

1. Enlaces de entrega

- **Repositorio de la solución** (incluye el nodo Pick & Place): `https://github.com/chaconsantiago966-create/ur5_ros2_moveit_lab`
- **Nodo Pick & Place** (paquete `ur5_pick_place`): `https://github.com/chaconsantiago966-create/ur5_ros2_moveit_lab/tree/main/src/ur5_pick_place`
- **Paquete generado con el MoveIt Setup Assistant** (`ur5_robotiq_moveit_config`): `https://github.com/chaconsantiago966-create/ur5_ros2_moveit_lab/tree/main/src/ur5_robotiq_moveit_config`
- **Nodo que ejecuta trayectorias con MoveIt** (`ur5_moveit_executor`): `https://github.com/chaconsantiago966-create/ur5_ros2_moveit_lab/tree/main/src/ur5_moveit_executor`
- **Paquetes base**: `https://github.com/DanielPalominoS/ur5_gripper_packages`

Tabla 1: Videos de evidencia (carpeta `media/videos` del repositorio).

Video	Contenido	Duración
`01_control_por_topicos`	Actividad 1: identificación de controladores y tópicos; brazo y gripper comandados con `ros2 topic pub`.	1:13
`02_pick_and_place`	Actividad 2: dos ciclos de la rutina Pick & Place con E/S digitales simuladas (Gazebo, RViz y log).	1:59
`03_setup_assistant_pasos`	Actividad 3: pasos de configuración en el MoveIt Setup Assistant.	1:00
`04a_moveit_rviz_error_aceleracion`	Actividad 3: falla inicial de planificación (límites de aceleración).	0:32
`04_moveit_rviz_planificacion`	Actividad 3: planificación y ejecución de los grupos `ur_arm` y `gripper` con el plugin de MoveIt en RViz.	1:03
`05_moveit_executor_gazebo`	Actividad 4: nodo MoveItPy ejecutando trayectorias en Gazebo y esquivando un obstáculo.	0:32

2. Objetivos

1. Construir los paquetes de simulación del UR5 con gripper en un *workspace* local y controlar el brazo y el gripper mediante comandos publicados en ROS.
2. A partir de los tópicos identificados, desarrollar un paquete con un nodo que simule una rutina *Pick & Place* con al menos tres puntos de interés y, en ellos, operaciones de apertura y cierre del gripper, mensajes informativos, espera de entradas digitales y activación de salidas digitales.
3. Generar con el *MoveIt Setup Assistant* un paquete de configuración de MoveIt para el robot con gripper, documentarlo y verificarlo planificando trayectorias en RViz.

4. Usar esa configuración para ejecutar trayectorias desde un nodo del ecosistema MoveIt.

3. Entorno y metodología

La solución se desarrolló sobre **ROS 2 Jazzy**, **Gazebo Harmonic** y **MoveIt 2.12**. La Tabla 2 resume las versiones. Como no fue posible instalar ROS 2 desde `packages.ros.org` en el equipo de ejecución (hallazgo H1), se utilizó la distribución **RoboStack** (conda-forge), que empaqueta las mismas versiones de ROS 2 Jazzy. La simulación se ejecutó en Ubuntu 24.04 sin GPU, con render por software (Mesa *llvmpipe*) sobre una pantalla virtual, desde la que se grabaron los videos. El repositorio incluye *scripts* para reproducir todo en Ubuntu 24.04 / WSL2 con la instalación estándar por `apt`.

Todos los resultados reportados se verificaron ejecutando la simulación; los *logs* completos se encuentran en `docs/logs`.

Tabla 2: Entorno de software.

Componente	Versión
Sistema operativo	Ubuntu 24.04 (x86_64, 2 núcleos, 8 GB RAM, sin GPU)
ROS 2	Jazzy (`ros-jazzy-desktop` 0.11, RoboStack)
Gazebo	Harmonic (`ros_gz` 1.0.24, `gz_ros2_control` 1.2.20)
MoveIt 2	2.12.4 (`moveit`, `moveit_py`, `moveit_setup_assistant`)
Universal Robots	`ur_description` 3.3, `ur_controllers` 3.9

Estructura del repositorio. La raíz del repositorio es el *workspace* de `colcon`:

```
ur5_ros2_moveit_lab/
  src/ur5_gripper_packages/       paquetes base (description + bringup), sin modificar
  src/ur5_pick_place/             Actividad 2: rutina Pick & Place + E/S simuladas
  src/ur5_robotiq_moveit_config/  Actividad 3: salida del MoveIt Setup Assistant
  src/ur5_moveit_executor/        Actividad 4: nodo MoveItPy
  scripts/  docs/  media/videos/  reporte/
```

Compilación.

```
bash scripts/instalar_dependencias.sh   # apt + rosdep
bash scripts/compilar.sh                # colcon build (sin --symlink-install, ver H2)
source scripts/entorno.sh
```

4. Actividad 1: simulación y control por tópicos

Se compilaron `ur5_robotiq_description` (ament_cmake) y `ur5_robotiq_bringup` (ament_python) y se lanzó `sim_gz_sgb.launch.py`, que inicia Gazebo, `robot_state_publisher`, el puente `/clock` y los controladores de `ros2_control`. La expansión del modelo con `xacro` y `check_urdf` confirmó la cadena cinemática `world` → `base_link_parent` → ... → `tool0` → {`gripper_base_link`, `tool0_tcp`}, donde `tool0_tcp` está 15 cm delante de la brida.

Con `ros2 control list_controllers`, `ros2 topic list` y `ros2 action list` se identificaron las interfaces de la Tabla 3.

Tabla 3: Interfaces de control identificadas.

Elemento	Interfaz	Tipo
Brazo	`/scaled_joint_trajectory_controller/joint_trajectory`	`trajectory_msgs/JointTrajectory`
Brazo	`/scaled_joint_trajectory_controller/follow_joint_trajectory`	acción `FollowJointTrajectory`
Gripper	`/gripper_controller/commands`	`std_msgs/Float64MultiArray`
Estado	`/joint_states`	`sensor_msgs/JointState`

Controladores activos:

- `joint_state_broadcaster`;
- `scaled_joint_trajectory_controller` (`ur_controllers/ScaledJointTrajectoryController`);
- `gripper_controller` (`position_controllers/JointGroupPositionController`), dos prismáticas de 0.002 a 0.042 m.

Ejemplos de comandos utilizados:

```
ros2 topic pub --once /scaled_joint_trajectory_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory "{joint_names: ['shoulder_pan_joint',
  'shoulder_lift_joint','elbow_joint','wrist_1_joint','wrist_2_joint','wrist_3_joint'],
  points: [{positions: [0.8, -1.2, 1.4, -1.77, -1.57, 0.0], time_from_start: {sec: 3}}]}"
ros2 topic pub --once /gripper_controller/commands std_msgs/msg/Float64MultiArray "{data: [0.004,
    0.004]}"
```

La lectura de `/joint_states` tras cada comando coincidió con las posiciones enviadas (error $< 10^{-9}$ rad en estado estacionario).

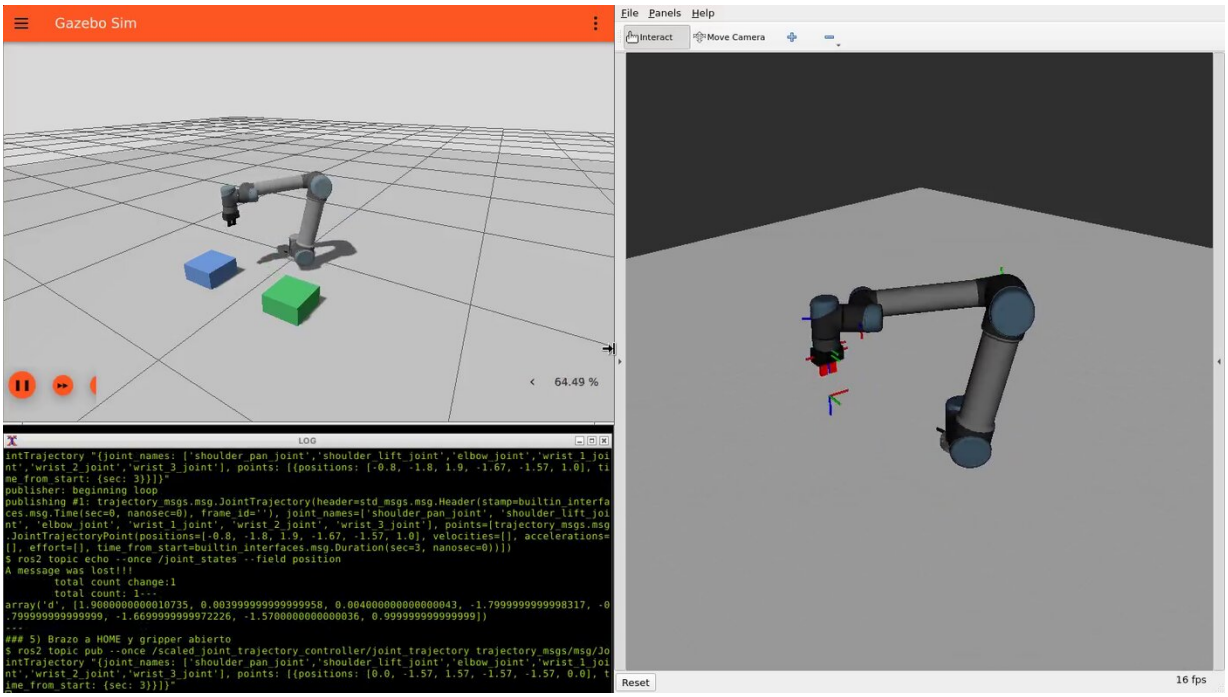


Figura 1: Control por tópicos: Gazebo (izq.), terminal con los comandos y lectura de `/joint_states`, y RViz (der.).

5. Actividad 2: rutina Pick & Place

5.1. Diseño

El paquete `ur5_pick_place` contiene dos nodos y un *launch* que reutiliza el bringup del repositorio base con un mundo propio (estaciones PICK en verde y PLACE en azul):

- **`digital_io_sim`** – simula el módulo de E/S de un controlador de robot o PLC con interfaces estándar de ROS 2: cada entrada y salida es un tópico `std_msgs/Bool` con QoS *transient local* (`/io/di0..3`, `/io/do0..3`) y se modifica con un servicio `std_srvs/SetBool` (`/io/di<n>/set`, `/io/do<n>/set`). Con `auto_cell:=true` emula la lógica de la celda; con `auto_cell:=false` las entradas se activan manualmente con `ros2 service call`.
- **`pick_place_node`** – intérprete de una rutina declarada en YAML. Mueve el brazo con la acción `FollowJointTrajectory` (espera el resultado; opcionalmente usa el tópico), acciona el gripper por `/gripper_controller/commands` verificando el resultado en `/joint_states`, publica mensajes en `/pick_place/status`, espera entradas, activa salidas y publica marcadores para RViz.

Tabla 4: Señales digitales simuladas.

Señal	Significado	Lógica automática de la celda
DO0	Banda transportadora	ON ⇒ a los 2 s DI0 = ON
DI0	Pieza presente en PICK	la activa la banda (o el operador)
DI2	Zona de descarga libre	inicia en ON
DO1	Pieza depositada (aviso al PLC)	ON ⇒ a los 1.5 s DI1 = ON y DI0 = OFF
DI1	Confirmación del PLC	la activa la lógica (o el operador)

5.2. Puntos de interés y operaciones

Los puntos se calcularon por cinemática inversa (parámetros DH del UR5, herramienta hacia abajo) y se verificaron en Gazebo con `tf2_echo world tool0_tcp` (p. ej. `pick` → (0.450, 0.250, 0.120) m, exacto). La Tabla 5 resume las operaciones realizadas en cada punto.

Tabla 5: Puntos de interés de la rutina y operaciones.

Punto	TCP (x, y, z) [m]	Operaciones
`home`	(0.49, 0.11, 0.28)	mensaje, abrir gripper, **activar DO0**, **esperar DI0**, desactivar DO0
`pick_approach`	(0.45, 0.25, 0.30)	mensaje
`pick`	(0.45, 0.25, 0.12)	**cerrar gripper**, mensaje “Pieza sujeta”
`inspect`	(0.00, 0.45, 0.35)	mensaje, pausa
`place_approach`	(0.45, −0.30, 0.30)	mensaje, **esperar DI2**
`place`	(0.45, −0.30, 0.12)	**abrir gripper**, **activar DO1**, **esperar DI1**, desactivar DO1

Fragmento de la rutina (`config/pick_place_routine.yaml`):

```
step_list:
  - "move:pick@2"
  - "gripper:close"
  - "message:Pieza sujeta"
  - "move:place_approach@3"
  - "wait_input:2=ON"
  - "move:place@2"
  - "gripper:open"
  - "set_output:1=ON"
  - "wait_input:1=ON"
```

5.3. Resultados

Se ejecutaron dos ciclos completos sin errores: 18 movimientos del brazo, 6 operaciones del gripper, 8 cambios de salidas digitales y 6 esperas de entradas satisfechas (log `docs/logs/pick_place_run.log`). En RViz la pieza (cubo naranja) aparece en PICK al activarse DI0, se adjunta al TCP al cerrar el gripper y queda en PLACE al abrirlo.

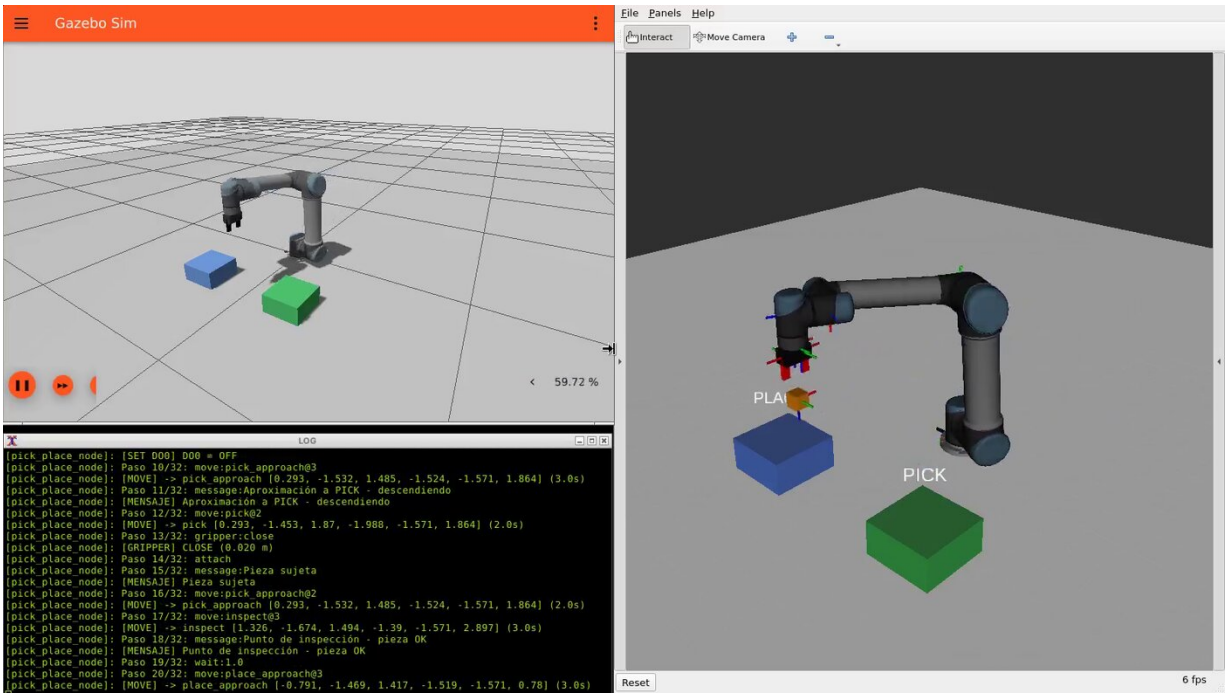


Figura 2: Pick & Place: Gazebo con las estaciones (izq.), log de la rutina (abajo) y RViz con la pieza sujeta camino a PLACE (der.).

```
[MENSAJE] Encendiendo banda transportadora (DO0)
[SET DO0] DO0 = ON
[WAIT DI0] esperando ON ...
[digital_io_sim]: DI0: OFF->ON
[WAIT DI0] condición cumplida
[MOVE] -> pick [0.293, -1.453, 1.87, -1.988, -1.571, 1.864] (2.0s)
[GRIPPER] CLOSE (0.020 m)
[MENSAJE] Pieza sujeta
```

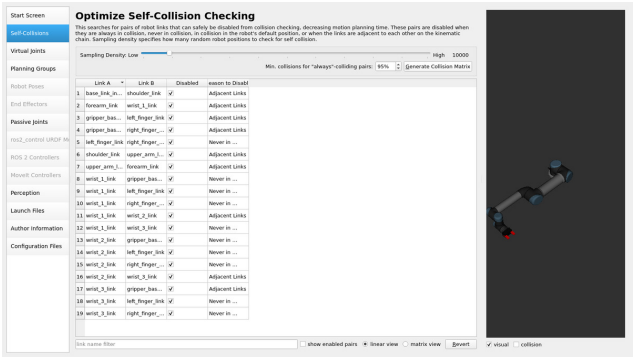
6. Actividad 3: paquete MoveIt con el Setup Assistant

6.1. Configuración

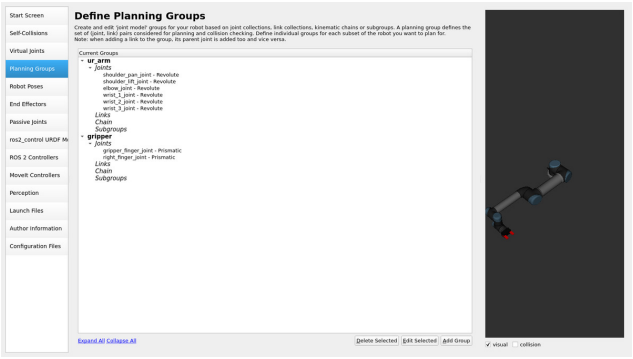
Se siguió la guía `MoveitSetupAssConfig.md` del repositorio base con la interfaz gráfica del asistente. La Tabla 6 resume cada paso y la Figura 3 muestra capturas; el video `03_setup_assistant_pasos` recorre todas las pantallas. La salida del asistente se guardó sin cambios en un *commit* propio, y las correcciones posteriores (H8–H9) en otro, para que sean trazables.

Tabla 6: Configuración realizada en el MoveIt Setup Assistant.

Pantalla	Configuración
Start Screen	Nuevo paquete desde `ur_with_gripper_moveit.urdf.xacro`
Self-Collisions	Matriz generada con valores por defecto: 19 pares deshabilitados
Virtual Joints	Ninguno: el URDF ya fija `world`→`base_link_parent` (H6)
Planning Groups	`ur_arm`: KDL, RRTConnect, 6 revolutas; `gripper`: sin solver, 2 prismáticas
Robot Poses	`home`, `ready` (`ur_arm`); `open` (0.04), `close` (0.004) (`gripper`)
End Effectors	`arm_eef`: grupo `gripper`, *parent link* `tool0` (H7)
ros2_control URDF	comando `position`; estado `position`, `velocity`
ROS 2 Controllers	`scaled_joint_trajectory_controller` (JointTrajectoryController), `gripper_controller` (GripperActionController)
MoveIt Controllers	`scaled_joint_trajectory_controller` (FollowJointTrajectory, por defecto), `gripper_controller` (GripperCommand)
Configuration Files	Salida en `src/ur5_robotiq_moveit_config`



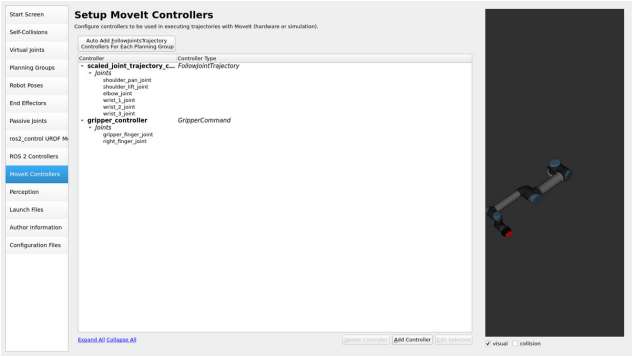
(a) Self-Collisions



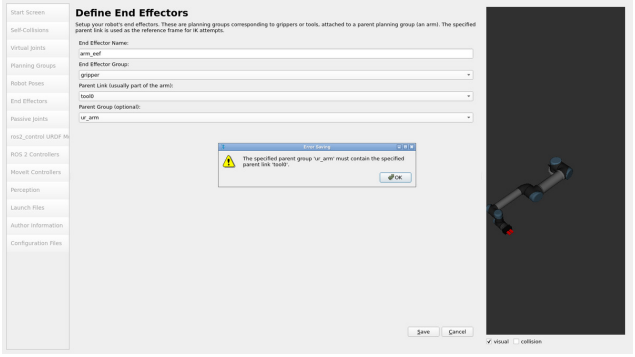
(b) Planning Groups



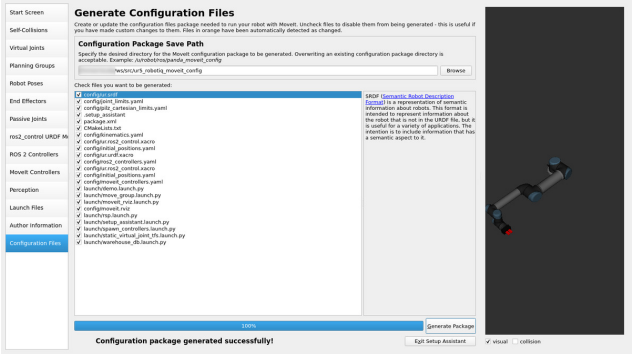
(c) Robot Poses



(d) MoveIt Controllers



(e) Error del *parent group* (H7)



(f) Paquete generado

Figura 3: Capturas del MoveIt Setup Assistant (todas en `docs/img/msa`).

6.2. Paquete obtenido

Tabla 7: Archivos principales de `ur5_robotiq_moveit_config`.

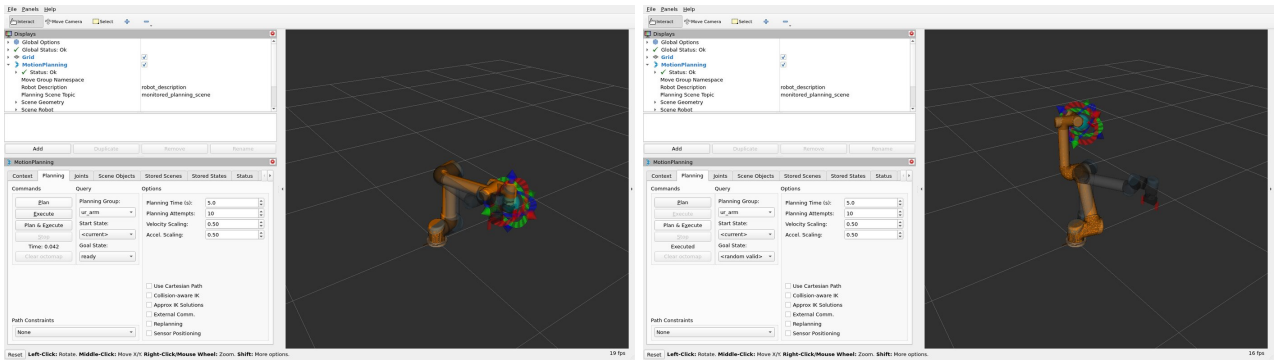
Archivo	Contenido
`config/ur.srdf`	grupos, estados con nombre, efector final y colisiones deshabilitadas
`config/kinematics.yaml`	`ur_arm` → `KDLKinematicsPlugin`
`config/joint_limits.yaml`	límites de velocidad y aceleración (corregido, H8)
`config/ros2_controllers.yaml`	controladores del sistema simulado del demo
`config/moveit_controllers.yaml`	controladores que MoveIt usa para ejecutar
`config/ur.urdf.xacro`, `ur.ros2_control.xacro`	URDF + `ros2_control` con `mock_components`
`launch/demo.launch.py`	RViz + `move_group` + `ros2_control` simulado

6.3. Pruebas de planificación en RViz

Con `ros2 launch ur5_robotiq_moveit_config demo.launch.py` se probaron ambos grupos desde el panel *MotionPlanning*. La primera ejecución falló en todas las planificaciones del brazo ("*No acceleration limit was defined for joint shoulder_pan_joint*", H8). Tras habilitar los límites de aceleración, todas las pruebas terminaron con estado `SUCCEEDED` (Tabla 8).

Tabla 8: Pruebas con el plugin de MoveIt en RViz.

Grupo	Meta	Comando	Resultado
`ur_arm`	`ready`	Plan + Execute	plan en 0.04 s, SUCCEEDED
`ur_arm`	`<random valid>`	Plan & Execute	SUCCEEDED
`ur_arm`	`home`	Plan & Execute	SUCCEEDED
`gripper`	`close` / `open`	Plan & Execute	SUCCEEDED (acción `gripper_cmd`)



(a) Plan a `ready` (estado meta en naranja) (b) Meta aleatoria válida

Figura 4: Planificación con el plugin MotionPlanning de RViz.

7. Actividad 4: ejecución de trayectorias con MoveIt

El paquete `ur5_moveit_executor` lanza Gazebo (bringup base), `move_group` con la configuración del asistente (cargada con `MoveItConfigsBuilder`), RViz y el nodo `moveit_executor`, escrito con la API de Python **MoveItPy**. El nodo:

1. planifica al estado `ready` del SRDF y lo ejecuta;
2. agrega un obstáculo ("pared" de $0{,}30 \times 0{,}04 \times 0{,}25$ m entre PICK y PLACE) a la *planning scene*;
3. planifica a una pose del TCP sobre PICK con OMPL/RRTConnect;
4. desciende y sube en línea recta con el planificador **Pilz LIN**;
5. cierra y abre el gripper (por tópico, ver H10);
6. planifica hasta PLACE esquivando la pared y regresa a `home`.

```
self.arm.set_goal_state(pose_stamped_msg=pose_goal(0.45, -0.30, 0.20), pose_link='tool0_tcp')
result = self.arm.plan()
status = self.robot.execute(result.trajectory, controllers=[])   # ExecutionStatus
```

Tabla 9: Resultados del nodo `moveit_executor` en Gazebo.

Meta	Planificador	Tiempo de plan	Puntos	Ejecución
Estado `ready`	OMPL	0.06 s	23	SUCCEEDED
Pose sobre PICK $(0{,}45, 0{,}25, 0{,}25)$	OMPL	0.06 s	7	SUCCEEDED
Descenso a $z = 0{,}12$	Pilz LIN	0.01 s	12	SUCCEEDED
Subida a $z = 0{,}30$	Pilz LIN	0.01 s	15	SUCCEEDED
Pose sobre PLACE (con pared)	OMPL	0.07 s	16	SUCCEEDED
Estado `home`	OMPL	0.08 s	14	SUCCEEDED

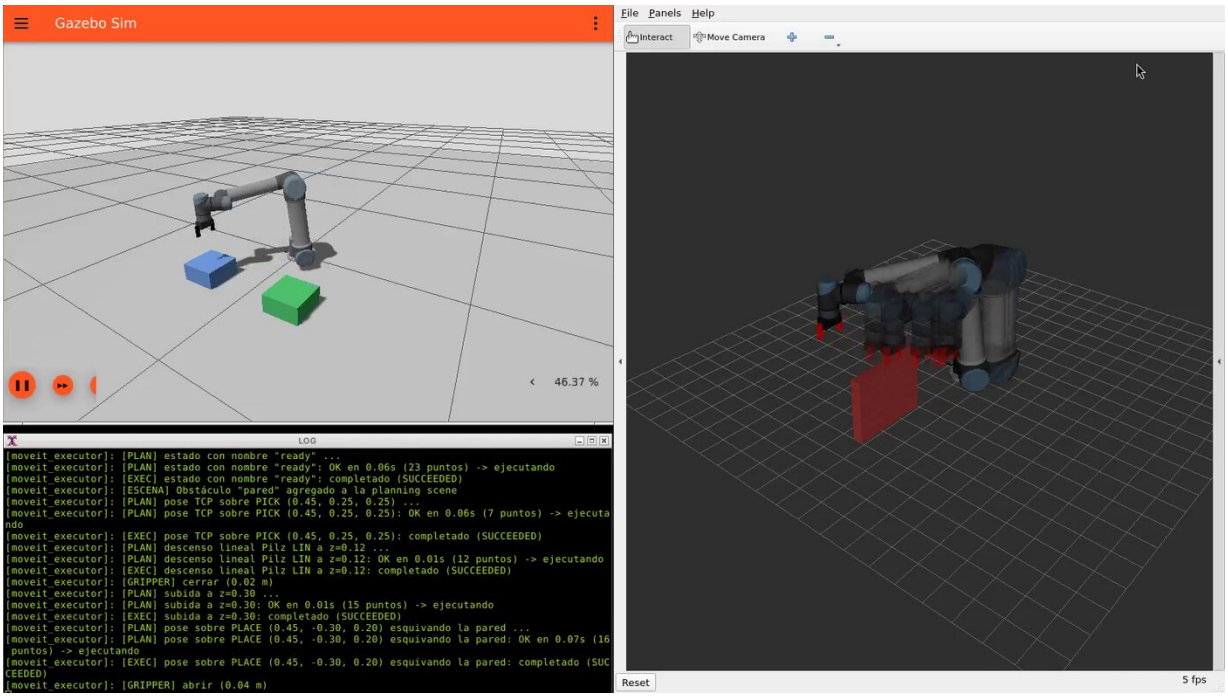


Figura 5: Nodo MoveItPy: el robot en Gazebo (izq.), log del nodo (abajo) y RViz con la pared en la escena de planificación y la trayectoria planificada (der.).

8. Hallazgos, dificultades y alternativas

El detalle completo, con mensajes de error y *logs*, está en `docs/hallazgos.md`.

#	Problema	Diagnóstico y solución / alternativa
H1	ROS 2 no instalable desde `packages.ros.org`	Host inaccesible (DNS/403). Alternativa: RoboStack (conda-forge) con ROS 2 Jazzy, Gazebo y MoveIt; en WSL se documenta la instalación por `apt`.
H2	`colcon build --symlink-install` falla	"*option –editable not recognized*": setuptools $\geq$ 80 eliminó el modo `develop`. Se compila sin `--symlink-install` (o `setuptools<80`).
H3	`xacro`: "*Undefined substitution argument name*"	El atributo `name` del robot usa `$(arg name)` antes de declararlo; pasar `name:=ur`.
H4	Tópicos `/io/di/0` inválidos	En ROS 2 un segmento no puede empezar con dígito; se usó `/io/di0`.
H5	El Setup Assistant se cierra al escribir la ruta	Recarga el archivo en cada tecla; si la ruta parcial es un directorio lanza `std::length_error` y aborta. Solución: pegar la ruta completa o usar *Browse*.
H6	Virtual joint de la guía	El URDF ya define `world`→`base_link_parent`; el asistente solo admite el link raíz como hijo. No se agregó.
H7	"*parent group must contain parent link tool0*"	`ur_arm` (definido por articulaciones) termina en `wrist_3_link`. Se dejó vacío el *parent group*, como en la guía; alternativa: definir el grupo como cadena hasta `tool0`.
H8	Planificación falla en `AddTimeOptimalParameterization`	El asistente genera `has_acceleration_limits: false`; se habilitaron 5 rad/s^2 (brazo) y 1 m/s^2 (dedos).

#	Problema	Diagnóstico y solución / alternativa
H9	Posiciones iniciales fuera de límites	Dedos en 0 con límite inferior 0.002 m; se usó HOME y gripper abierto.
H10	Controlador del gripper	`GripperActionController` maneja un solo joint; en Gazebo el gripper es un `JointGroupPositionController` sin acción. MoveIt ejecuta solo el brazo en Gazebo y el gripper se acciona por tópico. Alternativa futura: `mimic` + controlador de gripper paralelo.
H11	Estado inicial en colisión	En Gazebo el robot arranca con articulaciones en 0, dentro del obstáculo; se mueve a `ready` antes de insertarlo.
H12	`execute()` devuelve ABORTED (intermitente)	El cliente de acción aún no estaba conectado; se agregó espera, verificación de `ExecutionStatus` y reintentos.
H13	Obstáculo invisible en RViz	La escena de MoveItPy es local; se publica un *diff* en `/planning_scene`.
H14	Rendimiento sin GPU	RTF de Gazebo 45–65 % y RViz 3–30 fps con render por software; no afecta los resultados (tiempo de simulación).
H15	Detalles menores	Texto de RViz con espacios separados; clave duplicada en `.setup_assistant`; la pieza del Pick & Place es visual (sin física de agarre; alternativa: plugin `DetachableJoint`).

Fuentes consultadas. Documentación oficial de MoveIt 2 (tutoriales del Setup Assistant y de MoveItPy), documentación de `ros2_control`/`ros2_controllers`, la guía `MoveitSetupAssConfig.md` del repositorio base, y los mensajes de error y el código fuente de los paquetes involucrados.

9. Conclusiones

- Los controladores de `ros2_control` exponen interfaces simples (un tópico de trayectorias y una acción para el brazo, un tópico de posiciones para el gripper) que bastan para programar rutinas industriales sin planificador. Usar la acción `FollowJointTrajectory` en lugar del tópico permite saber cuándo termina cada movimiento y encadenar operaciones de forma determinista.
- Las E/S digitales se pueden simular con tópicos `Bool` y servicios `SetBool`, lo que permite probar la lógica de sincronización con la celda (esperas y confirmaciones) antes de tener el hardware, de forma automática o manual.
- El MoveIt Setup Assistant genera un paquete funcional en pocos minutos, pero su salida debe revisarse: en MoveIt 2 Jazzy faltan los límites de aceleración y las posiciones iniciales pueden quedar fuera de los límites. Además, la configuración de controladores debe coincidir con la que realmente expone la simulación.
- Con MoveItPy se combinan planificadores (OMPL para movimientos libres con evasión de obstáculos y Pilz LIN para aproximaciones rectilíneas), lo que reproduce la forma de programar un ciclo de *pick and place* en un robot real.